

✓ Support Vector Machine e Classificação de Sentimentos em Textos

Data de entrega: Até 12 de Maio de 2024

Como entregar: Enviar o seu jupyter notebook (.ipynb) pelo próprio Classroom. Uma cópia deste será automaticamente criada para você no seu Google Drive, você pode usar essa cópia mesmo, sem necessidade de criar outra.

Leia com atenção:

- Você deve fazer obrigatoriamente todas as atividades marcadas com a exclamação **!**.
- Atividades marcadas com **✖** são opcionais.
- Use o Google Colab para executar o notebook e fazer a tarefa, ou baixe no seu computador e faça na API que preferir, mas lembre-se de entregar no Classroom a versão final.
- A atividade vai ser baseada no seu número USP.
- Há questões dissertativas!
- Siga o passo a passo indicado no tutorial, respondendo as perguntas das atividades propostas no seu jupyter notebook.
- O notebook que entregar deve estar 100% funcional, isto é, todos os códigos rodando no Colab sem erros.
- Entregue o notebook completo, com todo conteúdo original e mais os seus códigos e textos.
- Esta tarefa é individual e haverá controle de similaridade. Tentativas de cópias, fraude, compra-venda de EPs não serão toleradas, levando a nota zero na tarefa e posterior análise para medidas cabíveis.

✓ **!Exercício!**

Para mostrar que você leu até aqui, escreva seu nome completo (assinatura) na linha a seguir:

Assinatura:

✓ Pre-requisitos

Primeiro vamos carregar algumas bibliotecas: Numpy, Scipy Linear Algebra (scipy.linalg), Pandas, System (sys) e Time (time)

```
import numpy as np
import scipy.linalg as la
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import matplotlib.cm as cm

import sys
import time
```

Agora vamos definir alguns números especiais ("magic") para cada aluno, com base no seu número USP. eles serão usados para personalizar a sua Atividade Computacional.

✓ **!Exercício!**

Preencha o seu número USP abaixo

```
# Coloque aqui o seu número USP
nusp = 10769448
```

Estes números serão importantes para a atividade:

```
nusp_str=str(nusp).replace("0","")

magic_ilong = int(nusp_str)
magic_ishort = int(str(magic_ilong)[:2])

# Ultimo dígito não nulo do nusp
magic_int = int(list(set(nusp_str))[-1])
```

```
print("magic_ilon = ", magic_ilon)
print("magic_ishort = ", magic_ishort)
print("magic_int = ", magic_int)
```

```
magic_ilon = 1769448
magic_ishort = 17
magic_int = 9
```

✓ Método de Newton multidimensional

Considere uma curva no plano, que podemos representar como uma curva de nível de uma função:

$$f(x, y) = 0$$

Por exemplo,

$$f(x, y) = x^2 + y^2 - c = 0$$

define um círculo no plano centrado no $(0, 0)$ com raio $\sqrt{c}$.

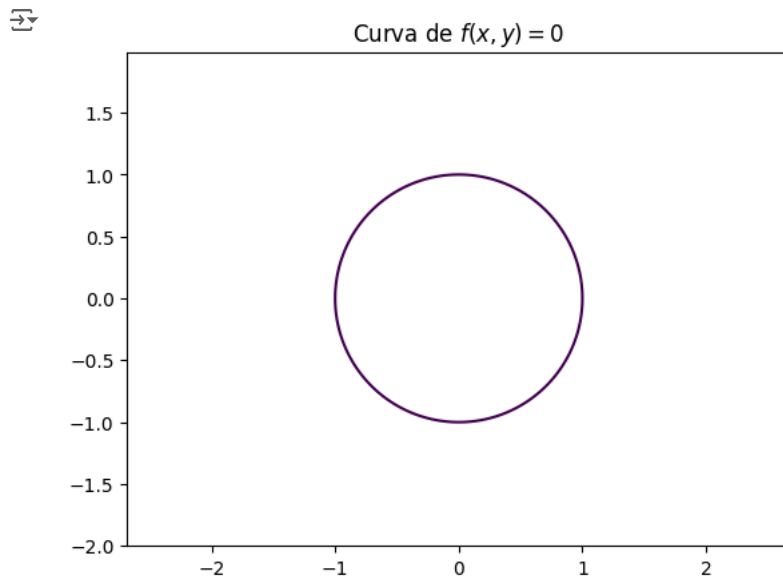
```
#Função
def f(x,y):
    # c=1
    return x**2+y**2 - 1.0

# Amostra de pontos no plano
x = np.arange(-2.0, 2.0, 0.01)
y = np.arange(-2.0, 2.0, 0.01)
X, Y = np.meshgrid(x, y)

# Função
Z = f(X,Y)

#Gráfico
fig, ax = plt.subplots()
CS = ax.contour(X, Y, Z, [0.0])
#ax.clabel(CS, inline=True, fontsize=10)
ax.set_title('Curva de $f(x,y)=0$')
ax.axis('equal')

plt.show()
```



Agora suponha que temos 2 curvas,

$$f_1(x, y) = 0$$

$$f_2(x, y) = 0$$

Por exemplo um círculo e uma elipse

$$f_1(x, y) = x^2 + y^2 - 1 = 0$$

$$f_2(x, y) = x^2/2 + 3y^2 - 2 = 0$$

```
def f1(x,y):
    return x**2+y**2 -1

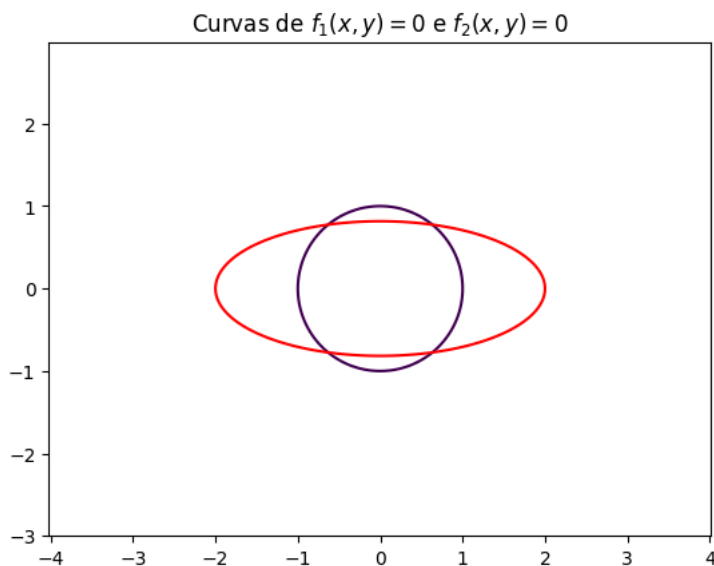
def f2(x,y):
    return x**2/2+3*y**2 - 2

# Amostra de pontos no plano
x = np.arange(-3.0, 3.0, 0.01)
y = np.arange(-3.0, 3.0, 0.01)
X, Y = np.meshgrid(x, y)

# Função
Zf1 = f1(X,Y)
Zf2 = f2(X,Y)

#Gráfico
fig, ax = plt.subplots()
CSf = ax.contour(X, Y, Zf1, [0.0])
CSg = ax.contour(X, Y, Zf2, [0.0], colors=('r'))
ax.set_title('Curvas de $f_1(x,y)=0$ e $f_2(x,y)=0$')
ax.axis('equal')

plt.show()
```



Como podemos fazer para achar os pontos de intersecção entre as curvas? Ou seja, um ponto (x, y) onde tanto f_1 quanto f_2 são zero?

Uma forma é usando o método de Newton, mas neste caso ele é bidimensional. Para facilitar a notação, escrevemos:

$$F(\vec{x}) = \begin{pmatrix} f_1(x, y) \\ f_2(x, y) \end{pmatrix}$$

onde $\vec{x} = (x, y)$. Logo, queremos encontrar um ponto $\vec{x}$ tal que $F(\vec{x}) = \vec{0}$, onde indicamos $\vec{0} = (0, 0)$.

Nessa notação, poderíamos considerar $\vec{x} \in \mathbb{R}^N$ e $F: \mathbb{R}^N \rightarrow \mathbb{R}^N$, e podemos escrever o método de Newton como

$$\vec{x}_{k+1} = \vec{x}_k - [F'(\vec{x}_k)]^{-1} F(\vec{x}_k),$$

sendo $\vec{x}_0$ dado, e F' representa a Jacobiana (matriz de derivadas $\partial F_i / \partial x_j$) de F .

Note que a cada iteração precisamos calcular uma inversa de uma matriz. Para evitar esse cálculo explícito, transformamos cada passo de Newton em um problema de resolver um sistema linear:

$$F'(\vec{x}_k) \vec{d}_k = -F(\vec{x}_k),$$

e depois calculamos

$$\vec{x}_{k+1} = \vec{x}_k + \vec{d}_k.$$

Vamos voltar ao nosso exemplo de calcular os pontos de intersecção de curvas de níveis. No exemplo do círculo e elipse, queremos resolver

$$F(\vec{x}) = \begin{pmatrix} x^2 + y^2 - 1 \\ x^2/2 + 3y^2 - 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

e a Jacobiana de F é dada por (verifique!)

$$F'(\vec{x}) = \begin{pmatrix} 2x & 2y \\ x & 6y \end{pmatrix}.$$

Para o método de Newton, a cada iteração precisamos resolver o seguinte sistema linear

$$\begin{pmatrix} 2x_k & 2y_k \\ x_k & 6y_k \end{pmatrix} \begin{pmatrix} dx_k \\ dy_k \end{pmatrix} = - \begin{pmatrix} x_k^2 + y_k^2 - 1 \\ x_k^2/2 + 3y_k^2 - 2 \end{pmatrix},$$

e atualizamos o passo fazendo

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} dx_k \\ dy_k \end{pmatrix} + \begin{pmatrix} x_k \\ y_k \end{pmatrix}.$$

✓ Exercício 1

Escreva um código em Python que encontre os pontos de intersecção de curvas de níveis no plano, com as seguintes características:

1. Considere como problema a ser resolvido o problema de intersecção do círculo e elipse com

$$F(\vec{x}) = \begin{pmatrix} x^2 + y^2 - 1 \\ x^2/M + 3y^2 - 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

onde M é o seu número "magic_int", definido com base no seu número USP (é o último dígito não nulo do seu número USP).

2. Escreva uma função que recebe um ponto (x, y) e calcula a $F(x, y)$ (note que ela recebe (x, y) e devolve um vetor em $\mathbb{R}^2$)
3. Escreva uma função que recebe um ponto (x, y) e calcula a matriz $F'(x, y)$.
4. Escreva uma função que recebe um ponto (x_k, y_k) e resolve o sistema linear $F'(x_k, y_k)\vec{d}_k = -F(x_k, y_k)$. Para a resolução do sistema linear, você pode implementar o seu próprio resolvidor, com eliminação de Gauss por exemplo, ou usar uma implementação do Python (Numpy/Scipy).
5. Seu código final deve começar com um chute inicial (x_0, y_0) e aplicar sucessivas iterações do método de Newton para encontrar os pontos de intersecção das curvas.
6. Cuidado! Você deve impor um número máximo de iterações e algum critério de parada, por exemplo quando $\|F(x, y)\| < 10e-6$. Caso em algum passo o sistema linear não tenha solução (determinante nulo), seu código deve retornar ao usuário um aviso (erro).

Com o seu código pronto, responda as perguntas:

1. Escreva as estimativas encontradas para os 4 pontos de intersecção das curvas.
2. Como fazemos para encontrar um ponto de intersecção ao invés de outro?
3. O método de Newton funciona para qualquer chute inicial? Sempre converge para um ponto de intersecção? Quando ele não funciona?

```
# F(x,y)
```

```
# F'(x,y)
```

```
# Sistema Linear
```

```
# Método de Newton
```

Respostas das perguntas:

- 1.
- 2.
- 3.

✓ Classificação de Dados

✓ Exemplo em dimensão 2

Suponha que temos informações da nota do ENEM de vários alunos que passaram na USP e na UNICAMP, por exemplo formando a tabela

Aluno	Onde passou	Nota Matemática	Nota Português
João	USP	9	7
José	USP	10	6
Maria	UNICAMP	7	8
Rosa	USP	10	4
Pedro	UNICAMP	3	7
Carlos	UNICAMP	6	9
Daniel	USP	9	5
Mario	UNICAMP	7	9
Ronaldo	USP	7	5
...	...	...	...

Seria possível montar algum modelo que permitir dizer onde um aluno que não está nessa tabela (novo aluno) passou olhando apenas as suas notas do ENEM?

Esse é um problema de classificação. Montamos um modelo com base nesses dados (treinamos um modelo com dados) e usamos esse modelo para classificar novos dados. Esse é um dos objetivo de um modelo que usa **Support Vector Machine** (SVM).

É claro que esse exemplo é fictício e simplificado, mas ele já nos fornece desafios suficientes para entendermos como funciona o SVM!

Vamos tentar entender o SVM com base nesse exemplo acima. Primeiro vamos fazer um gráfico das notas em relação às notas nas duas áreas, separando pelas duas categorias (universidades).

```
dados = {
    'aluno': ['João', 'José', 'Maria', 'Rosa', 'Pedro', 'Carlos', 'Daniel', 'Mario', 'Ronaldo', 'André', 'Gabriel', 'Nelson', 'Cláudia',
    'univ': ['USP', 'USP', 'UNICAMP', 'USP', 'UNICAMP', 'UNICAMP', 'UNICAMP', 'UNICAMP', 'UNICAMP', 'UNICAMP', 'UNICAMP', 'UNICAMP', 'UNICAMP',
    'nota_mat': [9, 10, 7, 10, 3, 6, 9, 7, 5, 3, 8, 9, 5, 6, 5],
    'nota_port': [7, 6, 8, 4, 7, 9, 5, 9, 5, 3, 9, 4, 3, 6, 5, 8]
}

df = pd.DataFrame(dados)

# Definir cores com base na variável 'local'
cores = {'USP': 'blue', 'UNICAMP': 'red'}

# Criar o gráfico de dispersão
plt.figure(figsize=(8, 6))
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

# Adicionar rótulos e título
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.axis([2, 10, 2, 10])

# Exibir o gráfico
print(df)
plt.show()
#print(df)
```

Queremos achar, inicialmente, uma reta que corta esse plano e separa as notas das duas universidades. Nem sempre isso é possível, mas nesse caso, parece que as notas da UNICAMP e da USP tem um certo agrupamento.

Vamos começar com um pouco de matemática. Para cada aluno (i) temos as seguintes informações:

- y_i : será -1 se for USP e $+1$ se for UNICAMP
- $\vec{x}_i$: um vetor de tamanho 2 com as notas de matemática e português, nessa ordem. No caso geral, esse vetor por ter tamanho arbitrário (chamaremos de p), com as características que nos interessam.

Logo, temos um conjunto para os n alunos dado por:

$$\begin{pmatrix} \vec{x}_1, y_1 \\ \vec{x}_2, y_2 \\ \vdots \\ \vec{x}_n, y_n \end{pmatrix}$$

com $\vec{x}_i \in \mathbb{R}^p$.

Objetivo: Achar a reta (ou hiperplano no caso geral) que divida os pontos $\vec{x}_i$ de forma a separar os grupos de pontos com base em $y_i = \pm 1$. Formalmente, queremos uma reta (hiperplano) que maximize a distancia entre ela e o ponto mais próximo de cada grupo.

Podemos escrever retas escolhendo valores a , b e c tais que

$$ax + by - c = 0,$$

ou, analogamente para hiperplanos, escolhendo vetores $w \in \mathbb{R}^p$

$$w^T \vec{x} - c = 0,$$

com $\vec{x} \in \mathbb{R}^p$ o vetor de coordenadas do $\mathbb{R}^p$

Vejamos no nosso exemplo, chutando valores de w e c :

```
plt.figure(figsize=(8, 6))

# Criar o gráfico de dispersão
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

#Reta (chutamos esses valores!)
# ax+by-c = 0, wx - c = 0, w= (a, b)
b = 3.5
a = -2.8
c = 2.0

# Fazendo o gráfico da reta usando 2 pontos da reta
x0 = 0
y0 = (c - a*x0)/b
x1 = 10
y1 = (c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='green', linestyle='--', label='Reta de Separação')

# Adicionar rótulos e título
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.axis([2, 10, 2, 10])
plt.show()
```

Será que esse chute foi uma boa escolha para separar os grupos? Como achar "a melhor reta"?

Se acharmos a tal reta, podemos usar ela para categorizar novos dados! Basta olhar se o dado está "acima" ou "abaixo" da reta e classificar ele como USP ou UNICAMP! Isso daria uma previsão de qual universidade o novo aluno deve ter ido. Essa previsão será melhor ou pior a depender do ajuste da nossa reta.

Note que qualquer reta de separação temos que os pontos da USP estão "abaixo" da reta, satisfazendo

$$ax + by - c < 0,$$

ou no caso geral

$$w^T \vec{x} - c < 0,$$

e os pontos da UNICAMP estão "acima" da reta, com

$$w^T \vec{x} - c > 0.$$

Para que possamos medir a qualidade do classificador, adicionamos uma margem de mais ou menos um a essa reta (hiperplano) e impomos que uma categoria deve satisfazer

$$w^T \vec{x}_i - c \geq 1,$$

para $i \in \text{UNICAMP}$, isto é, i tal que $y_i = +1$, e a outra com

$$w^T \vec{x}_i - c \leq -1,$$

para $i \in \text{USP}$, isto é, i tal que $y_i = -1$. Resumidamente, considerando os dois grupos, podemos escrever que

$$y_i(w^T \vec{x}_i - c) \geq 1.$$

No exemplo anterior, com a nossa reta chutada, as retas que definem as margem são $w^T \vec{x} - c = \pm 1$, como podemos ver na figura a seguir.

```
plt.figure(figsize=(8, 6))

# Criar o gráfico de dispersão
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

# Reta (chutada)
# ax+by-c = 0 ou wx - c = 0, w= (a, b)
b = 3.5
a = -2.8
c = 2.0

# Gráficos das retas
# 2 pontos da reta principal
x0 = 2
y0 = (c - a*x0)/b
x1 = 10
y1 = (c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='green', linestyle='--', label='Reta de Separação')

# 2 pontos da reta de margem
# ax+by-c = 1
x0 = 2
y0 = (1+c - a*x0)/b
x1 = 10
y1 = (1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem +1')

# 2 pontos da reta de margem
# ax+by-c = -1
x0 = 2
y0 = (-1+c - a*x0)/b
x1 = 10
y1 = (-1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem -1')

# Adicionar rótulos e título
plt.axis([2, 10, 2, 10])
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.show()
```

O método SVM irá buscar uma reta de separação/classificação de tal forma a separar os grupos e tentando aumentar a distância entre as margens! Isso é um problema de otimização, que vamos discutir a seguir.

Veja um exemplo onde tentamos aumentar a margem (distância entre as retas de suporte).

```
plt.figure(figsize=(8, 6))

# Criar o gráfico de dispersão
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

# Reta
# ax+by-c = 0
# wx - c = 0, w =(a,b)
b = 2.0
a = -1.6
c = 2.3

# 2 pontos da reta principal
x0 = 2
y0 = (c - a*x0)/b
x1 = 10
y1 = (c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='green', linestyle='--', label='Reta de Separação')

# 2 pontos da reta de margem
# ax+by-c = 1
x0 = 2
y0 = (1+c - a*x0)/b
x1 = 10
y1 = (1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem +1')

# 2 pontos da reta de margem
# ax+by-c = -1
x0 = 2
y0 = (-1+c - a*x0)/b
x1 = 10
y1 = (-1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem -1')

# Adicionar rótulos e título
plt.axis([2, 10, 2, 10])
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.show()
```

Nosso desafio no método de Support Vector Machine será o de achar a reta (hiperplano) que separa os pontos e tem a maior margem possível! Isto é, a distância entre as retas (hiperplanos) de suporte (onde não há dados) é a maior possível. Isto dá mais confiança ao classificador já que a reta (hiperplano) $\vec{w}^T x - c = 0$, ao longo da qual a classificação é duvidosa, está o mais longe possível dos dados verdadeiros. Isto é, um hiperplano com uma margem muito pequena poderia classificar um novo ponto como "vermelho" mesmo tendo um dado azul muito próximo dele, mas do lado oposto, o que provavelmente não corresponderia com a natureza verdadeira do novo ponto que estamos tentando classificar. Veja o ponto verde no gráfico abaixo, que seria classificado como um aluno da UNICAMP sendo que aparentemente ele tem mais semelhanças com as notas de um aluno da USP.


```
plt.figure(figsize=(8, 6))

# Criar o gráfico de dispersão
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

# Reta
# ax+by-c = 0, wx-c=0 com w= (a, b)
b = 5.5
a = -1.925
c = 22.275

# 2 pontos da reta principal
x0 = 2
y0 = (c - a*x0)/b
x1 = 10
y1 = (c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='green', linestyle='--', label='Reta de Separação')

# 2 pontos da reta de margem
# ax+by-c = 1
x0 = 2
y0 = (1+c - a*x0)/b
x1 = 10
y1 = (1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem +1')

# 2 pontos da reta de margem
# ax+by-c = -1
x0 = 2
y0 = (-1+c - a*x0)/b
x1 = 10
y1 = (-1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem -1')

# Ponto verde adicional, para exemplificar
plt.scatter([9.5], [7.6], color='green', alpha=0.5)

# Adicionar rótulos e título
plt.axis([2, 10, 2, 10])
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.show()
```

✓ !Exercício 2 !

A distância entre dois hiperplanos (retas) definidos por $\vec{w}^T \vec{x} - c = 1$ e $\vec{w}^T \vec{x} - c = -1$ pode ser calculado como

$$d(\vec{w}) = \frac{2}{\|\vec{w}\|},$$

onde $\|\vec{w}\|^2 = \vec{w}^T \vec{w} = \sum_{i=1}^p w_i^2$ é a norma 2 ao quadrado dos coeficientes que definem o hiperplano (reta).

No nosso exemplo no plano, temos $\vec{w} = (a, b)$, $\vec{x} = (x, y)$ e as retas

$$ax + by - c = \pm 1.$$

1. Verifique (demonstre) que no caso do plano realmente podemos calcular a distância entre as retas como $\frac{2}{\|(a,b)\|}$. Dica: as retas são paralelas e o vetor $\vec{w}$ é normal às retas!!
2. Faça um código que calcule a distância entre as retas. Calcule as distâncias entre as retas nos dois exemplos anteriores e confirme que o segundo exemplo tem margem maior que o primeiro.
3. Varie os valores de a , b e c e tente buscar algum caso onde a distância entre as retas aumenta e ainda conseguimos separar os dois grupos. É possível achar um hiperplano de separação mais "largo" que o ilustrado acima?

Coloque seu código aqui

Coloque as respostas/deduções/comentários.

- 1.
 - 2.
 - 3.
-

✓ Support Vector Machine

O algoritmo de Support Vector Machine (SVM) é um método usado em machine learning proposto na década de 1990 para fazer classificação supervisionada de dados. O método é supervisionado por precisar de um treinamento com uma base de dados para depois poder ser usado para classificar novos dados.

O método geral tem muitos detalhes técnicos, então aqui vamos discutir alguns casos mais específicos, com algumas suposições (depois vamos tentar relaxar algumas dessas hipóteses):

- Para a construção do modelo (treinamento), assumimos dados $\vec{x}_1, \dots, \vec{x}_n$, com $\vec{x}_n \in \mathbb{R}^p$, com respectivas classificações $y_1, y_2, \dots, y_n$, que para simplificar, vamos assumir $y_i = \pm 1$.
- Vamos supor que existe um hiperplano $w^T x - c = 0$ em $\mathbb{R}^p$ que separa os dados $\vec{x}_1, \dots, \vec{x}_n$. Isto é, os dados $\vec{x}_i$ que possuem a classificação $y_i = 1$ ficam de um dos lados do hiperplano enquanto os dados que possuem a classificação $y_i = -1$ ficam do outro lado.
- Vamos supor, sem perda de generalidade, que todos os pontos que estão "do lado de cima" do hiperplano, isto é, os $\vec{x}_i$ com $w^T \vec{x}_i - c > 0$ são tais que $w^T \vec{x}_i - c \geq 1$ (por que podemos fazer esta suposição sem perda de generalidade?). Do mesmo modo, os pontos que estão "do lado de baixo" do hiperplano são todos tais que $w^T \vec{x}_i - c \leq -1$. Sem perda de generalidade vamos supor que os vetores "do lado de cima" estão classificados com $y_i = +1$ e os do lado de baixo por $y_i = -1$.

Um bom classificador é aquele em que esta região que não possui dados, isto é, a região compreendida entre os hiperplano $w^T x - c = 1$ e $w^T x - c = -1$ é a maior possível. Utilizando a fórmula de distância entre dois (hiper)planos encontramos que esta distância é dada por:

$$\frac{2}{\sqrt{w^T w}}.$$

Nosso problema de otimização é, dentre todos os hiperplanos $w^T x - c = 0$ que classificam corretamente os dados nas condições descritas acima, encontrar aquele que maximiza a quantidade $\frac{2}{\sqrt{w^T w}}$. Convença-se que, de forma equivalente, podemos minimizar $\frac{w^T w}{2}$. Ou seja, o problema de interesse pode ser escrito como:

$$\begin{aligned} &\text{Minimizar}_{w \in \mathbb{R}^p, c \in \mathbb{R}} \quad \frac{1}{2} w^T w, \\ &\text{tal que} \quad y_i (w^T \vec{x}_i - c) \geq 1, i = 1, \dots, n. \end{aligned}$$

Ou seja, isto é o mesmo que dizer que quando $y_i = 1$, o ponto $\vec{x}_i$ tem que estar bem classificado acima do hiperplano ($w^T \vec{x}_i - c \geq 1$) e quando $y_i = -1$ o ponto $\vec{x}_i$ tem que estar bem classificado abaixo do hiperplano ($w^T \vec{x}_i - c \leq -1$).

Este é um problema de otimização quadrática convexa. A função objetivo a ser minimizada é uma função quadrática e convexa e as restrições são dadas por funções lineares das variáveis w e c . Este tipo de problema pode ser resolvido por uma variação do método de Newton que veremos abaixo.

Mais detalhes sobre SVMs podem ser encontrados aqui:

- <https://alex.smola.org/papers/2004/SmoSch04.pdf>

✓ Minimização com Método de Newton

Vamos relembrar o que sabemos dos cursos de cálculo a respeito do problema de minimizar uma função objetivo $f(z)$, $z \in \mathbb{R}^N$ com restrições $h_1(z) = 0, \dots, h_m(z) = 0$. Sob certas condições de regularidade e diferenciabilidade das funções, sabemos que em uma solução z^* deste problema o gradiente da função objetivo f em z^* é uma combinação linear dos gradientes das restrições em z^* . Isto é, existem os chamados multiplicadores de Lagrange $\lambda_1, \dots, \lambda_m \in \mathbb{R}$ tais que

$$\nabla f(z^*) + \sum_{i=1}^m \lambda_i \nabla h_i(z^*) = 0.$$

Quando não há restrição alguma no problema, esta condição se reduz a $\nabla f(z^*) = 0$. Quando tratamos de restrições de desigualdade, digamos, o problema de minimizar uma função $f(z)$, $z \in \mathbb{R}^N$, restrito a que algumas componentes de z sejam não negativas, isto é, $z_i \geq 0, i \in S$, onde $S \subseteq \{1, \dots, N\}$, pode-se mostrar que em uma solução z^* , a componente i do gradiente de f em z^* deve ser não-negativa quando $i \in S$ e $z_i^* = 0$, devendo se anular nos demais índices.

Por exemplo, o problema de minimizar a função $f(z) = (z + 1)^2$ restrita a $z \geq 0$ tem $z^* = 0$ como solução, e vale que a derivada $f'(z^*) = 2 \geq 0$, enquanto a função $f(z) = (z - 1)^2$ restrita a $z \geq 0$ tem $z^* = 1$ como solução e derivada $f'(z^*) = 0$.

```

t = np.arange(0.0, 5.0, 0.01)

plt.figure()

plt.subplot(221)
s = (t-1)*(t-1)

plt.plot(t, s)
plt.title('mínimo em $z>0$ (derivada nula)  ')

plt.subplot(222)
s = (t+1)*(t+1)

plt.plot(t, s)
plt.title('mínimo em $z=0$ (derivada não-negativa)')

plt.subplots_adjust(top=0.92, bottom=0.08, left=0.10, right=0.95, hspace=0.25,
                    wspace=0.35)

plt.show()

```

Então, em geral, podemos dizer que para o problema de minimizar $f(z)$, $z \in \mathbb{R}^N$ com a restrição que $z_i \geq 0$ para $i \in S \subseteq \{1, \dots, N\}$, se z^* é uma solução então vale que $\nabla f(z^*) = \mu$, onde $\mu_i \geq 0$, $i = 1, \dots, N$ com $\mu_i = 0$ se $i \notin S$ e $\mu_i z_i^* = 0$, $i \in S$. Isto é, em S , μ_i deve ser forçado em zero apenas quando $z_i^* > 0$, e no caso em que $z_i^* = 0$, não deve haver restrição adicional além de $\mu_i \geq 0$.

Combinando as duas ideias, considerando o problema:

$$\begin{aligned} &\text{Minimizar} && f(z), \\ &\text{tal que} && h_i(z) = 0, \quad i = 1, \dots, m, \\ &&& z_i \geq 0, \quad i \in S \subseteq \{1, \dots, N\}, \end{aligned}$$

pode-se mostrar que, sob certas condições, em uma solução z^* devem existir multiplicadores de Lagrange $\lambda_1 \in \mathbb{R}, \dots, \lambda_m \in \mathbb{R}$ e $\mu \in \mathbb{R}^N$ tais que (condições de Lagrange):

$$\begin{aligned} h_i(z^*) &= 0, \quad i = 1, \dots, m, \\ z_i^* &\geq 0, \quad i \in S, \\ \nabla f(z^*) + \sum_{i=1}^m \lambda_i \nabla h_i(z^*) &= \mu, \quad \mu \geq 0, \\ \mu_i &= 0, \quad i \notin S, \\ \mu_i z_i^* &= 0, \quad i \in S. \end{aligned}$$

A notação $\mu \geq 0$ ($\mu > 0$) significa que todas as componentes de μ são ≥ 0 (> 0). De volta ao nosso problema de classificação, vamos descrevê-lo da seguinte forma:

$$\begin{aligned} &\text{Minimizar}_{w \in \mathbb{R}^p, c \in \mathbb{R}, s \in \mathbb{R}^n} && \frac{1}{2} w^T w, \\ &\text{tal que} && y_i(w^T \vec{x}_i - c) - s_i = 1, \quad i = 1, \dots, n, \\ &&& s_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

Aqui simplesmente enxergamos cada desigualdade $g(z) \geq 1$ através de uma variável de folga s impondo $g(z) - s = 1$ e $s \geq 0$, o que descreve o problema de forma equivalente.

Neste caso, como as restrições são lineares nas variáveis (note que, no nosso problema, temos $z = (w, c, s) \in \mathbb{R}^p \times \mathbb{R} \times \mathbb{R}^n$), e a função objetivo é uma quadrática convexa, é conhecido que as condições de Lagrange descritas acima são necessárias e suficientes para que z^* seja uma solução do problema.

Para descrever o sistema de Lagrange, vamos utilizar as expressões descritas acima com $f(w, c, s) = \frac{1}{2} w^T w$ e $h_i(w, c, s) = y_i(w^T \vec{x}_i - c) - s_i - 1$, $i = 1, \dots, n$, considerando que das variáveis (w, c, s) , apenas s tem as restrições de sinal $s_i \geq 0$, $i = 1, \dots, n$. A parte central das condições de Lagrange consiste nas equações:

$$\begin{pmatrix} \nabla_w f(w, c, s) \\ \nabla_c f(w, c, s) \\ \nabla_s f(w, c, s) \end{pmatrix} + \sum_{i=1}^n \lambda_i \begin{pmatrix} \nabla_w h_i(w, c, s) \\ \nabla_c h_i(w, c, s) \\ \nabla_s h_i(w, c, s) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \mu \end{pmatrix}.$$

Note que um multiplicador $\mu \geq 0$ deve aparecer no lado direito apenas associado com as variáveis que estão sujeitas à restrição de sinal, ou seja, s . Para as demais variáveis o valor resultante deve ser zero, conforme explicado anteriormente.

Calculando as derivadas e complementando com as demais restrições, montamos o nosso sistema não linear de Lagrange conforme abaixo:

viabilidade primal:

$$(1) \quad y_i(w^T \vec{x}_i - c) - s_i - 1 = 0, i = 1, \dots, n, \\ s_i \geq 0, i = 1, \dots, n,$$

derivada com respeito a w :

$$(2) \quad w + \sum_{i=1}^n \lambda_i y_i \vec{x}_i = 0,$$

derivada com respeito a c :

$$(3) \quad -\sum_{i=1}^n \lambda_i y_i = 0$$

derivada com respeito a s (igual a μ):

$$(4) \quad -\lambda_j - \mu_j = 0, j = 1, \dots, n,$$

viabilidade dual:

$$\mu_j \geq 0, j = 1, \dots, n,$$

complementaridade:

$$(5) \quad \mu_j s_j - r = 0, j = 1, \dots, n, \text{ com } r = 0.$$

Note que a única não linearidade deste sistema consiste na restrição de complementaridade $\mu_j s_j = 0$. Esta restrição dá um sabor "combinatório" ao problema já que ela apenas diz que uma ou outra variável deve se anular. Esta restrição traz algumas dificuldades para a aplicação direta do método de Newton e por isso aliviamos este aspecto ao incorporar o parâmetro $r > 0$, transformando a equação em $\mu_j s_j - r = 0$. A ideia é reduzir r gradativamente para zero após cada iteração de Newton mantendo as variáveis μ_j e s_j estritamente positivas (ou seja, "interiores"). Isto funciona melhor na prática do que aplicar Newton diretamente e está associado a uma teoria rica dos chamados métodos de pontos interiores. Em particular, pode-se provar que o número de iterações necessárias para encontrar uma solução com precisão $\varepsilon > 0$ é da ordem de $\sqrt{n} \log\left(\frac{1}{\varepsilon}\right)$.

Sendo mais específico, vamos aplicar o método de Newton a este sistema de equações (e inequações) usando a estratégia dos conhecidos **Métodos de Pontos interiores** para minimizar funções com restrições, da seguinte forma:

1. Primeiro, tomamos um chute inicial $w_0 \in \mathbb{R}^p$, $c_0 \in \mathbb{R}$ e $0 < s_0 \in \mathbb{R}^n$ para as variáveis primais e $\lambda_0 \in \mathbb{R}^n, 0 < \mu_0 \in \mathbb{R}^n$ para as variáveis duais.
2. O valor r inicial será considerado como um parâmetro e será inicializado como $r = \frac{\mu_0^T s_0}{n} > 0$, que será reduzido após cada iteração de Newton ($r := \gamma r$, para algum $0 < \gamma < 1$).
3. Dada uma aproximação $z_k \in \mathbb{R}^N$ para a solução do sistema não linear $F(z) = 0 \in \mathbb{R}^N$, as iterações do método de Newton consistem em resolver o sistema linear $F'(z^k)d = -F(z^k)$, onde $F'(z^k)$ é a matriz Jacobiana de F em z^k , e em seguida atualizar $z^{k+1} = z^k + d$. No nosso problema, as variáveis são $z^k = (w^k, c^k, s^k, \lambda^k, \mu^k)$. Da mesma forma, $d = (d_w, d_c, d_s, d_\lambda, d_\mu)$.
4. Precisamos nos lembrar das restrições de desigualdade $\mu \geq 0$ e $s \geq 0$. Para isso, vamos corrigir a atualização das variáveis utilizando a expressão $z^{k+1} = z^k + td$ para algum $t > 0$ que garanta $s^{k+1} > 0$ e $\mu^{k+1} > 0$. Para isso, vamos tomar t como sendo o maior número tal que $s^k + td_s \geq 0$ e $\mu^k + td_\mu \geq 0$. A atualização é feita então com $z^{k+1} = z^k + 0.99td$.

Escrevendo o sistema de Newton para o nosso problema, temos as equações abaixo:

derivada de (1)

$$\begin{pmatrix} y_1 \vec{x}_1^T & -y_1 & & & \\ \vdots & \vdots & & & \\ y_n \vec{x}_n^T & -y_n & & & \end{pmatrix} \begin{pmatrix} d_w \\ d_c \\ d_s \\ d_\lambda \\ d_\mu \end{pmatrix} = \begin{pmatrix} -y_1(w^T \vec{x}_1 - c) + s_1 + 1 \\ \vdots \\ -y_n(w^T \vec{x}_n - c) + s_n + 1 \end{pmatrix}$$

derivada de (2) e (3)

$$\begin{pmatrix} I & 0 & 0 & y_1 \vec{x}_1 & \dots & y_n \vec{x}_n & 0 \\ 0 & 0 & 0 & -y_1 & \dots & -y_n & 0 \end{pmatrix} \begin{pmatrix} d_w \\ d_c \\ d_s \\ d_{\lambda_1} \\ \vdots \\ d_{\lambda_n} \\ d_\mu \end{pmatrix} = \begin{pmatrix} -w - \sum_{i=1}^n \lambda_i y_i \vec{x}_i \\ \sum_{i=1}^n \lambda_i y_i \end{pmatrix}$$

derivada de (4)

$$\begin{pmatrix} 0 & 0 & 0 & -I & -I \end{pmatrix} \begin{pmatrix} d_w \\ d_c \\ d_s \\ d_\lambda \\ d_\mu \end{pmatrix} = \lambda + \mu$$

derivada de (5)

$$\begin{pmatrix} 0 & 0 & \begin{bmatrix} \mu_1 & & \\ & \ddots & \\ & & \mu_n \end{bmatrix} & 0 & \begin{bmatrix} s_1 & & \\ & \ddots & \\ & & s_n \end{bmatrix} \end{pmatrix} \begin{pmatrix} d_w \\ d_c \\ d_s \\ d_\lambda \\ d_\mu \end{pmatrix} = \begin{pmatrix} -\mu_1 s_1 + r \\ \vdots \\ -\mu_n s_n + r \end{pmatrix}$$

Note que este é um sistema linear com $3n + p + 1$ equações e $3n + p + 1$ variáveis. Matricialmente, temos

$$\begin{pmatrix} YX & -y & -I & 0 & 0 \\ I & 0 & 0 & X^T Y^T & 0 \\ 0 & 0 & 0 & -y^T & 0 \\ 0 & 0 & 0 & -I & -I \\ 0 & 0 & M & 0 & S \end{pmatrix} \begin{pmatrix} d_w \\ d_c \\ d_s \\ d_\lambda \\ d_\mu \end{pmatrix} = \begin{pmatrix} -YXw + y^T e c + s + e \\ -w - \lambda^T X^T Y^T \\ \lambda^T y \\ \lambda + \mu \\ -\mu^T s + r \end{pmatrix},$$

onde e é o vetor de um's, X é a matriz $n \times p$ onde cada linha é um ponto $\vec{x}_i$ da amostra, Y é a matriz diagonal com o vetor y na diagonal, M é a matriz diagonal com o vetor μ na diagonal e S é a matriz diagonal com o vetor s na diagonal.

Referência:

- <https://www.maths.ed.ac.uk/~gondzio/reports/jpmXXV.pdf>

✓ Nossa Implementação

```
def F(X, y, w, c, lam, mu, s, r):
    # Recebe:
    # X - matrix (n x p) com cada linha uma amostra/ponto diferente (x_i), e cada ponto tendo dimensão "p".
    # y - vetor (n x 1) com classificação dos pontos +-1 para cada ponto
    # w - vetor (p x 1) com os pesos para a equação do hiperplano
    # c - constante do hiperplano
    # lam - vetor (n x 1) dos multiplicadores de Lagrange (variável auxiliar)
    # mu - vetor (n x 1) multiplicadores (variável auxiliar)
    # s - vetor (n x 1) vetor de variáveis de folga (variável auxiliar)
    # r - escalar auxiliar
    #
    # Devolve o vetor da função F(z) que queremos achar o zero.
    # Observação z=[w,c,s,lam,mu]
    # Vetor : F(z)

    #Matriz de dados
    n, p = X.shape

    y = y.reshape(n,1)

    yr = np.repeat( y, p, axis=1 )
    YX = np.multiply(yr, X)

    lamYXt = np.dot(lam.T, YX).T
    wtX = np.dot(w.T, X.T).reshape(n,1)
    lamy = np.dot(lam.T, y).reshape(1, 1)

    # F(z)
    Fz = np.block([
        [y*(wtX-c)-s-1],
        [w+lamYXt],
        [-lamy],
        [-lam-mu],
        [mu*s-r]
    ])

    # Retorna o valor da função que queremos achar o zero.
    return Fz
```

```

def sistema_newton(X, y, w, c, lam, mu, s, r):
    # Recebe:
    # X - matrix (n x p) com cada linha uma amostra/ponto diferente (x_i), e cada ponto tendo dimensão "p".
    # y - vetor (n x 1) com classificação dos pontos +-1 para cada ponto
    # w - vetor (p x 1) com os pesos para a equação do hiperplano
    # c - constante do hiperplano
    # lam - vetor (n x 1) dos multiplicadores de Lagrange (variável auxiliar)
    # mu - vetor (n x 1) multiplicadores (variável auxiliar)
    # s - vetor (n x 1) vetor de variáveis de folga (variável auxiliar)
    # r - escalar auxiliar
    #
    # Devolve Solução do sistema do método de Newton com direções na ordem
    # [dw,
    #  dc,
    #  ds,
    #  dlam,
    #  dmu]

    # Observação z=[w,c,s,lam,mu]
    # Matriz : F'(z)
    # Vetor : F(z)
    # onde F' é a Jacobiana do problema de Newton e F é a função que queremos encontrar o zero.
    # O sistema linear resolvido é F'(z) d = - F(z)

    #Montar a matrix para o método de Newton
    n, p = X.shape

    y = y.reshape(n,1)

    yr = np.repeat( y, p, axis=1 )
    YX = np.multiply(yr, X)

    Onxn = np.zeros((n,n))
    Opxn = np.zeros((p,n))
    Onxp = np.zeros((n,p))
    Opx1 = np.zeros((p,1))
    O1xp = np.zeros((1,p))
    O1xn = np.zeros((1,n))
    Onx1 = np.zeros((n,1))
    Inxn = np.eye(n)
    Ipxp = np.eye(p)
    M = np.diag(mu.reshape(n))
    S = np.diag(s.reshape(n))

    #Matrix
    A=np.block([
        [YX, -y, -Inxn, Onxn, Onxn],
        [Ipxp, Opx1, Opxn, YX.T, Opxn],
        [O1xp, 0, O1xn, -y.T, O1xn],
        [Onxp, Onx1, Onxn, -Inxn, -Inxn],
        [Onxp, Onx1, M, Onxn, S ]
    ])

    #Resolve sistema linear usando Numpy
    z = np.linalg.solve(A, -F(X, y, w, c, lam, mu, s, r))

    dw = z[:p]
    dc = z[p]
    ds = z[p+1:p+1+n]
    dlam = z[p+1+n:p+1+2*n]
    dmu = z[p+1+2*n:p+1+3*n]

    # Retorna solução do sistema linear (incremento do passo de Newton)
    return dw, dc, ds, dlam, dmu

```

```

# Método de pontos interiores

def pontos_interiores_SVM(X, y, tolresid = 1e-12, itmax = 1000):
    #
    # Recebe:
    # X : matriz nxp de dados
    # y : vetor nx1 de classificações +-1
    #
    # Opcionais:
    # tolresid : critério de parada (tolerância do resíduo - valor de ||F(z)||)
    # itmax : máximo de iterações
    #
    # Retorna:
    # w : vetor de definição do hiperplano
    # c : constante associada ao hiperplano de separação
    #

    #Inicialização - dimensões
    n, p = X.shape

    #Chute inicial para hiperplano (zero)
    w = np.zeros(p).reshape(p,1)
    c = 0

    # Chute inicial das variáveis auxiliares

    # Vetor de folga
    s = np.ones(n).reshape(n,1)

    #Multiplicadores
    mu = np.ones(n).reshape(n,1)
    lam = np.zeros(n).reshape(n,1)

    # Parâmetro inicial de controle da complementaridade
    r = np.matmul(s.T,mu)/n

    # Loop principal do Método

    #Iteração
    resid = 1
    it = 0
    while (resid>tolresid) and (it<itmax):
        it=it+1

        # Resolver o sistema linear de Newton
        dw, dc, ds, dlam, dmu = sistema_newton(X, y, w, c, lam, mu, s, r)

        # Imposição das restrições de desigualdades em s e mu
        t_s = 1.0
        if np.any(ds<0):
            t_s = 0.99*np.min(-s[ds<0]/ds[ds<0])
        t_mu = 1.0
        if np.any(dmu<0):
            t_mu = 0.99*np.min(-mu[dmu<0]/dmu[dmu<0])
        t = np.min([t_s, t_mu])

        # Atualizar as variáveis
        w=w+t*dw
        c=c+t*dc
        s=s+t*ds
        lam=lam+t*dlam
        mu=mu+t*dmu

        # Atualizar o parâmetro r
        r=0.2*r

        #Cálculo do resíduo no sistema original (com r=0)
        resid = np.linalg.norm(F(X, y, w, c, lam, mu, s, 0))

    print()
    print("Iteração: ", it, " Resíduo:", resid)
    print(" Hiperplano w:", w.T, " c:", c)

    print()
    print("Solução final. Hiperplano w:", w.T, " c:", c)
    return w, c

```

Vamos testar o código com os nossos dados, mas primeiro vamos colocar eles no formato que discutimos na parte do método de pontos interiores, com a matriz X e o vetor y.

```
dados = {
    'aluno': ['João', 'José', 'Maria', 'Rosa', 'Pedro', 'Carlos', 'Daniel', 'Mario', 'Ronaldo', 'André', 'Gabriel', 'Nelson', 'Cláudia',
    'univ': ['USP', 'USP', 'UNICAMP', 'USP', 'UNICAMP', 'UNICAMP', 'USP', 'UNICAMP', 'USP', 'USP', 'UNICAMP', 'USP', 'USP', 'UNICAMP',
    'nota_mat': [9, 10, 7, 10, 3, 6, 9, 7, 7, 5, 3, 8, 9, 5, 6, 5],
    'nota_port': [7, 6, 8, 4, 7, 9, 5, 9, 5, 3, 9, 4, 3, 6, 5, 8]
}

df = pd.DataFrame(dados)
x1 = np.array(df['nota_mat'].values)
x2 = np.array(df['nota_port'].values)

# Transformar nomes das universidade em números +-1
df.loc[df["univ"] == "USP", "univ_num"] = -1
df.loc[df["univ"] == "UNICAMP", "univ_num"] = +1
y = np.array(df['univ_num'].values)

# Matriz X
X = np.array((x1, x2)).transpose()
n, p = X.shape

# Vetor de classificação y
y = y.reshape(n,1)

# Teste do método de pontos interiores
w, c = pontos_interiores_SVM(X, y)
```

Vejamos graficamente como ficou a solução encontrada.

```
plt.figure(figsize=(8, 6))

# Definir cores com base na variável 'local'
cores = {'USP': 'blue', 'UNICAMP': 'red'}

# Criar o gráfico de dispersão
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

#Reta
# wx - c = 0, com w= (a, b)
a = w[0]
b = w[1]
#2 pontos da reta principal
x0 = 2
y0 = (c - a*x0)/b
x1 = 10
y1 = (c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='green', linestyle='--', label='Reta de Separação')

#2 pontos da reta de margem
# wx - c = 1, com w= (a, b)
x0 = 2
y0 = (1+c - a*x0)/b
x1 = 10
y1 = (1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem +1')

#2 pontos da reta de margem
# wx - c = -1, com w= (a, b)
x0 = 2
y0 = (-1+c - a*x0)/b
x1 = 10
y1 = (-1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem -1')

# Adicionar rótulos e título
plt.axis([2, 10, 2, 10])
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.show()
```


O que achou da solução? Graficamente, parece realmente que é o hiperplano com a maior margem possível. Veja que a margem "toca" alguns pontos (dados), estes pontos são chamados de vetores de suporte (**support vectors**). No nosso exemplo, são os pontos (5,6) ou (7,8) para a margem superior, e o ponto (6,5) para a inferior.

No caso em que os dados não podem ser separados, o modelo precisa incorporar erros, ou seja, o classificador deve ser capaz de classificar errado alguns dos pontos dados. Mas isto deve ocorrer apenas quando estritamente necessário. Um modelo que considera esta possibilidade é o seguinte:

$$\begin{aligned} \text{Minimizar}_{w \in \mathbb{R}^p, c \in \mathbb{R}, \varepsilon \in \mathbb{R}^n} \quad & \frac{1}{2} w^T w + C \sum_{i=1}^n \varepsilon_i, \\ \text{tal que} \quad & y_i (w^T \vec{x}_i - c) \geq 1 - \varepsilon_i, i = 1, \dots, n, \\ & \varepsilon_i \geq 0, i = 1, \dots, n. \end{aligned}$$

A variável ε_i permite que o hiperplano classifique errado o dado $\vec{x}_i$, mas o parâmetro $C \gg 0$ força que isto aconteça o mínimo possível. Em particular, quando os dados são separáveis, um C grande o suficiente força que todos os ε_i sejam iguais a zero, recuperando o modelo anterior.

✖ Exercício ✖

(Opcional)

Implemente um método de pontos interiores que considera o modelo acima. Para tanto, você deve incluir as variáveis ε_i no sistema não linear de Lagrange, usando as mesmas técnicas ilustradas anteriormente.

Teste o seu código com o exemplo anterior, permitindo que agora pequenos erros sejam aceitáveis, e compare com os resultados anteriores.

Coloque seu código aqui

Coloque as respostas/deduções/comentários aqui.

✔ Implementação "caixa preta"

O propósito dessa atividade do curso é entendermos como funciona um SVM, analisando uma possível implementação, via otimização usando o método dos pontos interiores. Por outro lado, várias bibliotecas oferecem implementações de algoritmos de SVM, então vamos ver como uma dessas bibliotecas se sai em comparação com o nosso algoritmo.

Usaremos uma biblioteca chamada

- **scikit-learn: Machine Learning in Python**

Ela possui um módulo de SVM, que pode ser chamado via **import sklearn.svm**.

<https://scikit-learn.org/stable/modules/classes.html#module-sklearn.svm>

Vamos usar uma função que resolve o problema de otimização proposto acima chamada **LinearSVC**. A implementação deles sempre leva em conta a "regularização" descrita acima, permitindo pequenos erros de classificação, por isso precisamos definir o parâmetro C como sendo muito grande para imitarmos o nosso algoritmo. Com a função "loss" escolhida como "hinge" o problema resolvido é exatamente o mesmo que apresentamos acima. A diferença é que ele usa um algoritmo que explora melhor a estrutura do problema (em particular, o problema "dual" é resolvido).

Vejam uma implementação.

```

from sklearn.svm import LinearSVC

#Dados (os mesmos de antes)
n, p = X.shape
x_train = X
y_train = y.reshape(n)

# Primeiro dizemos que tipo de modelo que queremos (SVM linear)
# Colocamos C bem grande, para que ele não permita erros de classificação
# a função loss é escolhida como 'hinge' para coincidir com o modelo apresentado
model = LinearSVC(loss='hinge',C=1e12, max_iter=10000)

# Agora pedimos para ele ajustar o modelo, ou seja,
# resolver o problema de otimização e achar o hiperplano
svmfit = model.fit(x_train,y_train)

# Esses são os parâmetros do hiperplano ajustado pelo sklearn
w_sklearn = svmfit.coef_[0]

# **Cuidado**, o nosso c é -c do sklearn!!!
c_sklearn = - svmfit.intercept_

print(" w , c ")
print(w_sklearn, c_sklearn)

```

Veja como o resultado do w e c ficou similar ao nosso! Vejamos graficamente.

```

plt.figure(figsize=(8, 6))

# Definir cores com base na variável 'local'
cores = {'USP': 'blue', 'UNICAMP': 'red'}

# Criar o gráfico de dispersão
for local, cor in cores.items():
    plt.scatter(df[df['univ'] == local]['nota_mat'], df[df['univ'] == local]['nota_port'], label=local, color=cor, alpha=0.5)

#Reta
# wx - c = 0, com w= (a, b)
a = w_sklearn[0]
b = w_sklearn[1]

#2 pontos da reta principal
x0 = 2
y0 = (c - a*x0)/b
x1 = 10
y1 = (c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='green', linestyle='--', label='Reta de Separação')

#2 pontos da reta de margem
# wx - c = 1, com w= (a, b)
x0 = 2
y0 = (1+c - a*x0)/b
x1 = 10
y1 = (1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem +1')

#2 pontos da reta de margem
# wx - c = -1, com w= (a, b)
x0 = 2
y0 = (-1+c - a*x0)/b
x1 = 10
y1 = (-1+c - a*x1)/b
plt.plot([x0, x1], [y0, y1], color='red', linestyle=':', label='Reta de Margem -1')

# Adicionar rótulos e título
plt.axis([2, 10, 2, 10])
plt.xlabel('Nota Matemática')
plt.ylabel('Nota Português')
plt.legend(title='Universidade')
plt.show()

```

✓ !Exercício 3 !

Considerando este problema bidimensional de classificação, com esses dados anteriores, faça as seguintes atividades.

1. Verifique o que acontece com o número de iterações do método de pontos interiores quando o parâmetro r é inicializado como zero.
2. Na implementação do método de pontos interiores, reduzimos o parâmetro r em 80% a cada iteração de Newton ($r := \gamma r$, com $\gamma = 0.2$). Estude o comportamento do algoritmo variando o parâmetro γ . Para isso, faça uma tabela do número de iterações em função de γ para algumas escolhas de γ .
3. Faça um critério de parada mais refinado deixando o algoritmo continuar iterando mesmo quando o critério de parada relativo ao resíduo é atingido. Utilize como critério de parada adicional que a norma da diferença entre duas iterações $\|z^{k+1} - z^k\|$ é menor que 10^{-12} .
4. A equação (4) define que o vetor μ é igual a $-\lambda$. Isso significa que podemos reduzir a dimensão do sistema que precisamos resolver, eliminando uma das variáveis λ ou μ . Copie os códigos anteriores e ajuste a implementação do método de pontos interiores eliminando a variável λ do problema. Teste o seu método e verifique se a solução coincide com a implementação original.

Coloque seus códigos aqui

Comentários/respostas aqui:

- 1.
- 2.
- 3.
- 4.

Trabalhando com textos

Um primeiro desafio de trabalhar com textos é o de como transformar palavras e frases em algo numérico para tratarmos no computador. No que segue vamos discutir uma forma simples de fazer isso, conhecido como "Bag of Words" (BOW), que em português trduzimos como "Saco de Palavras".

Resumidamente, um BOW define uma matriz que relaciona um vocabulário (conjunto de palavras) e conta quantas vezes cada palavra do vocabulário aparece em cada frase, resultando em algo como:

Frases	azul	branco	brilhar	céu	nuvem	sol
O céu está azul	1	0	0	1	0	0
As nuvens estão brancas	0	1	0	0	1	0
O sol está brilhando	0	0	1	0	0	1

Vamos ver como construir uma BOW?

Referências:

- <https://www.kaggle.com/code/mehmetlaudekman/text-classification-svm-explained/notebook>
- https://edisciplinas.usp.br/pluginfile.php/6288975/mod_resource/content/0/Aula%209%20-%20NLTK%20e%20spaCy.pdf

Bag of Words

Começaremos com um conjunto de frases para exemplificar.

```
frases = [  
    "O céu está azul",  
    "As nuvens estão brancas",  
    "O sol está brilhando",  
    "Não há ganho sem dor"  
]
```

Para o **Bag of Words**, precisamos pré-processar as frases para considerar apenas palavras "chaves". Vamos fazer as seguintes "limpezas":

- Remover caracteres que não sejam letras (pontuação). Para isso vamos usar Regular Expression ('re') e simplesmente tirar a pontuação das strings.
- Desconsideramos as palavras de conexão, conhecidas como **stopwords**, que incluem artigos, pronomes, conjunções, etc. Por exemplo: "de", "a", "um", "para". Para fazer isso, vamos usar uma base de stopwords de Português da biblioteca Natural Language Toolkit (NLTK).
- Fazer uma **lematização**, isto é, reduzir a palavra à sua forma canônica, levando em conta sua classe gramatical. Ex: estudar: estudando, estudioso, estudei. Nesta parte usaremos a biblioteca SpaCy (Industrial-Strength Natural Language Processing).

Os primeiros dois passos são bem simples, bastando consultar bases de dados existentes de português para saber que símbolos e palavras retirar da frase. A parte de Lematização é bem complicada. Para saber mais sobre Lematização e as bibliotecas SpaCy e NLTK, veja o material a seguir:

https://edisciplinas.usp.br/pluginfile.php/6288975/mod_resource/content/0/Aula%209%20-%20NLTK%20e%20spaCy.pdf

```
# Importar bibliotecas de texto
#-----

# Trabalhar com expressões regulares
import re

# Vamos usar a biblioteca NLTK e a biblioteca spaCy para pre-processar textos

import nltk # Natural Language Toolkit
import spacy # Industrial-Strength Natural Language Processing

#Palavras de junção, como artigos, preposições, pronomes e conjunções
from nltk.corpus import stopwords
nltk.download('stopwords')
nltk.download('punkt')
swords = nltk.corpus.stopwords.words('portuguese')
swords.remove("não")

# spaCy - baixar modelos de processamento de texto para Português
# Será usado para a "lematização"
#!python -m spacy download pt_core_news_sm #modelo pequeno
#!python -m spacy download pt_core_news_md # médio
nlp = spacy.load("pt_core_news_md")
#!python -m spacy download pt_core_news_lg #Grande

# Contador de texto do SKLEARN - scikit-learn Machine Learning in Python
from sklearn.feature_extraction.text import CountVectorizer

#Palavras de conexão / stopwords
print("Stopwords:", swords)

# Função que pré-processa frases, considerando:
# - Remover caracteres que não sejam letras (pontuação).
# Para isso vamos usar Regular Expression ('re') e simplesmente tirar a pontuação das strings.
# - Desconsideramos as palavras de conexão, conhecidas como **stopwords**,
# que incluem artigos, pronomes, conjunções, etc. Por exemplo: "de", "a", "um", "para".
# Para fazer isso, vamos usar uma base de stopwords de Português da biblioteca Natural Language Toolkit (NLTK).
# - Fazer uma **lematização**, isto é, reduzir a palavra à sua forma canônica,
# levando em conta sua classe gramatical. Ex: estudar: estudando, estudioso, estudei.
# Nesta parte usaremos a biblioteca SpaCy (Industrial-Strength Natural Language Processing).
#

def limpa_frase(frase):

    # Pega apenas letras - inglês
    #text = re.sub("[^a-zA-Z]", " ", frase)

    # Tira pontuação
    text = re.sub(r"[^\w\s]", " ", frase)

    # Tokenizing - Separa cada palavra como um elemento de uma lista
    # converte tudo em minúsculo
    text = nltk.word_tokenize(text.lower())

    # Removing stopwords - tira palavras de conexão - swords define a lingua
    text = [word for word in text if word not in swords]

    # Joining - volta para ser uma frase
    text = " ".join(text)

    #Lemmatizing - agrupa palavras por sua raiz - inglês
    #text = [lemma.lemmatize(word) for word in text]

    #Lemmatizing - agrupa palavras por sua raiz - portugues
    doc = nlp(text)
    text = [token.lemma_ for token in doc if not token.is_punct]

    # Joining - volta para ser uma frase
    text = " ".join(text).lower()

    return text
```

Vamos fazer um teste simples do nosso limpador de frase. Notem como ele mudar as palavras para sua forma "canônica", com base na sua raiz, neste exemplo com "pequenas" virando "pequeno", "coisas" virando "coisa", ...

```
frase = "As pequenas coisas na vida são as que mais nos trazem felicidade."  
print(frase)  
print(limpa_frase(frase))
```

Agora vamos passar o limpador em todas as nossas frases.

```
frases_limpo = []  
  
for frase in frases:  
    frases_limpo.append(limpa_frase(frase))  
  
print(frases)  
print(frases_limpo)
```

Quais são as palavras que mais aparecem nas nossas frases? Isso é uma noção de **Vocabulário** usado. Poderíamos fazer um código simples em Python para contar isso, mas ele ficaria lento para rodar em bases grandes. Então vamos usar códigos existentes.

```
# Inicialize o vetorizador CountVectorizer, para contar palavras  
vectorizer = CountVectorizer()  
  
# Alimenta o vetorizador aos dados (construa o vocabulário)  
vectorizer.fit(frases_limpo)  
vocab = vectorizer.get_feature_names_out()  
print("Vocabulário:", vocab)
```

Finalmente podemos explicar o que é um Bag of Words!!! Ele é simplesmente uma matriz que cruza nossas frases (linhas) com nosso vocabulário (colunas), indicando quantas vezes cada palavra do vocabulário aparece na frase. Vejamos no exemplo:

```
#Cria o bag_of_words, cruzando as frases e suas contagens em relação ao vocabulário  
bag_of_words = vectorizer.fit_transform(frases_limpo)  
  
# Converta a matriz de bag of words para um DataFrame Pandas  
df_bag_of_words = pd.DataFrame(bag_of_words.toarray(), columns=vocab)  
  
# Adicione uma coluna para representar as frases  
df_bag_of_words['Frases'] = frases  
  
# Reorganize as colunas para ter as frases na primeira coluna  
colunas = ['Frases'] + list(vocab)  
df_bag_of_words = df_bag_of_words[colunas]  
  
# Imprima a matriz de bag of words  
print("Bag of Words:")  
print(df_bag_of_words)
```

Importante

Esse método de Bag-of-words é interessante, mas tem algumas limitações, entre elas:

- Note que a matriz gerada ignora a posição da palavra na frase. Então frases distintas podem ter a mesma representação na matriz. Por exemplo,
 - "o mar não está para peixe", e
 - "no mar tem muito peixe",
 - seriam iguais em um vocabulário apenas com "mar" e "peixe".
- Ele não leva em conta o contexto e sua conexão com outras palavras da frase. Por exemplo:
 - em "o gato mia, o cachorro late",
 - miar está associado ao gato, e latir ao cachorro, mas o BOW não será capaz de capturar isso.
- Não leva em conta a frequência que a palavra ocorre em textos.
- Quando usado em um modelo de aprendizado de máquina, ele não consegue achar representação para novas palavras que o seu vocabulário nunca viu.

Por outro lado, trata-se de um método interessante e simples para termos como primeiro método em um curso de métodos numéricos! Ele é a base para entendermos métodos mais sofisticados.

✓ Sentimentos

Agora vamos pensar que cada frase tem a si associado uma emoção, um sentimento. Nesta atividade vamos considerar apenas sentimentos positivos (coisas boas) e negativos (coisas ruins). Exemplos de frases:

```
frases_positivas = [  
    "Cada desafio é uma oportunidade.",  
    "O amor está em todos os lugares.",  
    "A esperança nos mantém.",  
    "Um sorriso ilumina o dia.",  
    "A gratidão transforma.",  
    "Empatia nos conecta.",  
    "Paz interior é valiosa.",  
    "Persistência leva ao sucesso.",  
    "Generosidade enriquece.",  
    "Compaixão une corações.",  
    "Amizade é um tesouro.",  
    "Otimismo ilumina o caminho.",  
    "Coragem supera medo.",  
    "Humildade nos ensina.",  
    "Perdão liberta.",  
    "Aceitação traz paz.",  
    "Criatividade é expressão pura.",  
    "Resiliência nos fortalece.",  
    "Fé nos sustenta.",  
    "Beleza está em toda parte.",  
    "Amor é poderoso.",  
    "O Amor é poderoso.",  
    "Simplicidade traz felicidade.",  
    "Inspiração está ao redor.",  
    "Esperança nos eleva.",  
    "Alegria contagia.",  
    "Gentileza é presente.",  
    "Sucesso é autodescoberta.",  
    "Harmonia traz equilíbrio.",  
    "Inspirar é transformar.",  
    "Autoconfiança é vital.",  
    "Positividade ilumina.",  
    "Fé nos guia.",  
    "Resiliência nos levanta.",  
    "Abundância é gratidão.",  
    "Determinação é poder.",  
    "Compreensão une.",  
    "Confiança fortalece.",  
    "Liberdade é essencial.",  
    "Sabedoria é guia.",  
    "Imperfeição é bela.",  
    "Generosidade nutre.",  
    "Autenticidade é vital.",  
    "Paixão nos move.",  
    "Jornada é felicidade.",  
    "Coragem é enfrentar medos.",  
    "Alegria está na jornada.",  
    "Paciência é virtude.",  
    "Gratidão é chave.",  
    "Generosidade é conexão.",  
    "Autenticidade é essencial.",  
    "A vida é cheia de possibilidades emocionantes!",  
    "Sempre há algo para apreciar em cada dia.",  
    "O amor e a bondade estão ao nosso redor, se prestarmos atenção.",  
    "Cada desafio é uma oportunidade para crescer e aprender.",  
    "A esperança é a luz que guia nossos corações em tempos difíceis.",  
    "O sorriso é a linguagem universal da felicidade.",  
    "A gratidão transforma o que temos em suficiente e mais.",  
    "A empatia nos conecta uns aos outros em um nível mais profundo.",  
    "A paz interior é o verdadeiro tesouro da vida.",  
    "A persistência nos leva ao sucesso, passo a passo.",  
    "A generosidade nos enriquece mais do que qualquer riqueza material.",  
    "A compaixão é o alicerce da verdadeira harmonia entre os seres humanos.",  
    "A amizade é um tesouro inestimável que valorizamos ao longo da vida.",  
    "O otimismo nos permite ver o mundo com cores mais brilhantes.",  
    "A coragem nos ajuda a enfrentar nossos medos mais profundos.",  
    "A humildade nos torna receptivos às lições que a vida nos oferece.",  
    "O perdão liberta nosso coração do peso do ressentimento.",  
    "A aceitação nos permite abraçar a nós mesmos e aos outros com amor incondicional.",  
    "A criatividade é a expressão da alma em sua forma mais pura.",  
    "A resiliência nos permite superar os momentos mais difíceis da vida.",  
    "A fé nos dá força para continuar, mesmo quando tudo parece perdido.",  
    "A beleza está em toda parte, se apenas abirmos os olhos para vê-la.",  
    "O amor é a força mais poderosa do universo, capaz de curar todas as feridas.",  
    "A simplicidade nos lembra que a verdadeira felicidade reside nas coisas mais simples da vida.",  
    "A inspiração está sempre ao nosso redor, se apenas estivermos dispostos a vê-la.",  
    "A esperança nos dá asas para voar, mesmo quando estamos cercados por nuvens escuras.",  
    "A alegria contagiosa deixa uma marca indelével em todos os corações que toca.",  
    "A gentileza é um presente que podemos dar a nós mesmos e aos outros a qualquer momento.",  
    "O sucesso é a jornada de autodescoberta que nos leva além de nossos limites autoimpostos.",  
    "A harmonia é a sinfonia da vida, onde cada nota contribui para a melodia da existência.",  
]
```


"A inspiração é a centelha que acende o fogo da criatividade dentro de nós.",
 "A autoconfiança é o alicerce sobre o qual construímos nossos sonhos mais grandiosos.",
 "A positividade é o elixir da vida que nos mantém radiantes, mesmo nos dias mais sombrios.",
 "A esperança é a estrela-guia que nos guia através das trevas e nos mostra o caminho para a luz.",
 "A paciência é a virtude que nos permite suportar as tempestades da vida com calma e serenidade.",
 "A gratidão é a chave que abre as portas para uma vida cheia de amor e abundância.",
 "A determinação é o motor que nos impulsiona a alcançar as estrelas, mesmo quando todos os outros desistirem.

]

```
frases_negativas = [
    "Às vezes, a vida parece esmagadora.",
    "Decepções podem abalar nossa confiança.",
    "O medo pode nos paralisar.",
    "Solidão é uma sombra constante.",
    "A tristeza envolve o coração em trevas.",
    "Raiva consome nossos pensamentos.",
    "Ansiedade cria uma prisão mental.",
    "A dor é uma inevitável companheira.",
    "Incerteza gera desconforto.",
    "A culpa é uma âncora pesada.",
    "Dúvida mina nossa confiança.",
    "Arrependimento assombra escolhas passadas.",
    "Preocupação nos consome.",
    "A Preocupação consome.",
    "Inveja envenena o coração.",
    "A amargura corrói de dentro para fora.",
    "Negatividade se espalha como fogo.",
    "Desconfiança quebra relacionamentos.",
    "Decepção abala a confiança.",
    "Desespero nos afunda.",
    "Apatia rouba nossa paixão.",
    "Remorso nos assombra.",
    "Negação evita a realidade.",
    "Solidão é um oceano sem fim.",
    "Confusão nos deixa perdidos.",
    "Mágoa nos fecha.",
    "Insegurança paralisa.",
    "Desconhecido ameaça.",
    "Perda deixa um vazio.",
    "Sobrecarga nos sufoca.",
    "Rotina nos aprisiona.",
    "Ciclo vicioso nos prende.",
    "Ignorância dói.",
    "Mal compreendido nos isola.",
    "Traição nos machuca.",
    "Sufocamento nos esmaga.",
    "Subestimação nos diminui.",
    "Rejeição dói.",
    "Julgamento nos fere.",
    "Abandono nos assusta.",
    "Desapontamento nos entristece.",
    "Exclusão nos isola.",
    "Incompreensão nos confunde.",
    "Ridículo nos envergonha.",
    "Menosprezo nos deprecia.",
    "Subjugação nos oprime.",
    "Engano nos irrita.",
    "Manipulação nos faz sentir impotentes.",
    "Às vezes, a vida pode parecer esmagadora e desanimadora.",
    "As decepções podem nos fazer questionar nosso próprio valor.",
    "O medo pode nos paralisar e impedir de alcançar nossos objetivos.",
    "A solidão pode ser uma companheira sombria nos momentos mais difíceis.",
    "A tristeza pode envolver nosso coração em uma nuvem escura.",
    "A raiva pode consumir nossos pensamentos e nos tornar cegos para o amor ao nosso redor.",
    "A ansiedade pode criar uma prisão em nossa mente, limitando nossa liberdade.",
    "A dor é uma parte inevitável da vida, mas também pode ser uma oportunidade para crescer.",
    "A incerteza pode criar um senso de desconforto e insegurança.",
    "A culpa pode pesar como uma âncora em nossa alma, nos impedindo de seguir em frente.",
    "A dúvida pode minar nossa confiança e nos impedir de alcançar nossos sonhos.",
    "O arrependimento pode nos assombrar, lembrando-nos de escolhas passadas que não podemos desfazer.",
    "A preocupação constante pode nos consumir, roubando-nos a alegria do momento presente.",
    "A inveja pode envenenar nossos corações, nos fazendo desejar o que os outros têm.",
    "A amargura pode nos corroer por dentro, transformando-nos em versões sombrias de nós mesmos.",
    "A negatividade pode se espalhar como uma praga, infectando tudo ao nosso redor.",
    "A desconfiança pode corroer relacionamentos, minando a base da confiança mútua.",
    "A decepção pode nos fazer questionar se podemos confiar em alguém novamente.",
    "O desespero pode nos fazer sentir como se estivéssemos presos em um abismo sem fim.",
    "A apatia pode roubar nossa paixão pela vida, deixando-nos vazios por dentro.",
    "O remorso pode nos perseguir, lembrando-nos de erros passados que gostaríamos de esquecer.",
    "A negação pode nos impedir de enfrentar a realidade, mantendo-nos presos em uma ilusão de segurança.",
    "A solidão pode nos fazer sentir como se estivéssemos perdidos em um mar de rostos desconhecidos.",
    "A confusão pode nos deixar perplexos e incertos sobre qual caminho seguir.",

```

```

"A mágoa pode criar uma barreira entre nós e os outros, nos isolando em nossa própria dor."

# Revisando nossas frases:
for i in range(len(frases)):
    if sentimentos[i] > 0:
        sent = "positivo"
    else:
        sent = "negativo"
    print(frases[i], sentimentos[i], sent)

```

Vamos passar nosso limpador em todas as nossas frases. Precisamos garantir que não haja frases repetidas, ou contradições (mesma frase com sentidos opostos). Então, além de limpar, vamos reduzir as duplicações.

```

frases_limpo = []
sentimentos_limpo = []
for frase, sent in zip(frases, sentimentos):
    frase_limpa = limpa_frase(frase)
    if frase_limpa not in frases_limpo:
        print("+ Frase: ", frase, " Limpa:", frase_limpa, " Sentimento:", sent)
        frases_limpo.append(frase_limpa)
        sentimentos_limpo.append(sent)
    else:
        print("- Excluindo a frase repetida:", frase, " Limpa:", frase_limpa, " Sentimento:", sent)

```

Vamos criar um **vocabulário** unificado, para os dois conjuntos de frases, positivo e negativo.

```

# Inicialize o vetorizador CountVectorizer, para contar palavras
vectorizer = CountVectorizer(max_features=10000)

# Alimenta o vetorizador aos dados (construa o vocabulário)
vectorizer.fit(frases_limpo)
vocab = vectorizer.get_feature_names_out()
print("Vocabulário:", vocab)

```

Vamos criar o BAG-OF-WORDS:

```

#Cria o bag_of_words, cruzando as frases e suas contagens em relação ao vocabulário
bag_of_words = vectorizer.fit_transform(frases_limpo)

# Converta a matriz de bag of words para um DataFrame Pandas
df_bag_of_words = pd.DataFrame(bag_of_words.toarray(), columns=vocab)

# Adicione uma coluna para representar as frases
df_bag_of_words['Frases'] = frases_limpo

# Reorganize as colunas para ter as frases na primeira coluna
colunas = ['Frases'] + list(vocab)
df_bag_of_words = df_bag_of_words[colunas]

# Imprima a matriz de bag of words
print("Bag of Words:")
print(df_bag_of_words)

```

Lembrando que para cada frase temos um sentimento, positivo ou negativo, isso define a base de dados que queremos modelar!

```

# Adicione uma coluna para representar as frases
df_bag_of_words['Sentimentos'] = sentimentos_limpo

# Reorganize as colunas para ter uma ordem boa para apresentação dos dados
colunas = ['Frases'] + ['Sentimentos'] + list(vocab)
df_bag_of_words = df_bag_of_words[colunas]

# Imprima a matriz de bag of words
print("Bag of Words:")
print(df_bag_of_words)

```